03

PARTIE 3 • RAPPEL ET CONSOLIDATION

Les structures de contrôle itératives

Ce que tu sauras faire à la fin

- Choisir la bonne boucle : `Pour`, `Répéter` ou `Tant que`
- Écrire correctement un compteur, un accumulateur et un drapeau booléen
- Contrôler la saisie d'une donnée avec `Répéter … Jusqu'à`
- Parcourir une chaîne ou un vecteur du début à la fin, ou à l'envers
- Repérer et éviter les boucles infinies

Pré-requis

- Les structures conditionnelles (partie 2)
- Les vecteurs et les chaînes (partie 1)
- L'affectation et les opérateurs

Mots-clés

Pour · range · while · Répéter · compteur · accumulateur · condition d'arrêt

Durée conseillée : 4 séances

I La boucle complète : Pour … Faire

DÉFINITION

La structure itérative **complète** `Pour … Faire` est utilisée lorsque le nombre de répétitions est **connu à l'avance**.

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`Pour <compteur> de <Vi> à <Vf> Faire`
`  <traitement>`
`Fin Pour`
	`for compteur in range(Vi, Vf+1) :`
	`  <traitement>`

REMARQUE

- Le compteur est de type **scalaire énuméré** (entier ou caractère) et son pas vaut 1 par défaut.
- Il est initialisé à Vi et prend la valeur suivante après chaque répétition, jusqu'à atteindre Vf.
- Vi et Vf doivent être de même type que le compteur.
- `range(Vf)` : la valeur initiale est 0 et Vf est **exclue**.
- `range(Vi, Vf, P)` : le pas devient P (P peut être négatif).

Écriture Python	Valeurs prises par i
`range(5)`	`0 · 1 · 2 · 3 · 4`
`range(1, 5)`	`1 · 2 · 3 · 4`
`range(0, 10, 2)`	`0 · 2 · 4 · 6 · 8`
`range(4, -1, -1)`	`4 · 3 · 2 · 1 · 0`
`range(len(ch))`	`0 … long(ch) − 1 : parcours d'une chaîne`

ERREUR FRÉQUENTE

`range(1, 5)` ne contient **pas** 5. Pour aller de Vi à Vf **inclus**, il faut écrire `range(Vi, Vf + 1)`.

ACTIVITÉ — 1 — INVERSER UNE CHAÎNE

Soit la séquence algorithmique suivante :

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`ch2 ← ""`
`Pour i de 0 à long(ch)-1 Faire`
`  ch2 ← ch[i] + ch2`
`Fin Pour`
	`ch2 = ""`
	`for i in range(len(ch)) :`
	`  ch2 = ch[i] + ch2`

1) Exécute à la main cette séquence pour `ch = "ALGORITHME"`.

`ch2 =` ***"EMHTIROGLA"***

2) Déduis-en le rôle de cette séquence.

Cette séquence inverse la chaîne : ch2 contient les caractères de ch écrits de droite à gauche

(le dernier caractère de ch devient le premier de ch2).

3) Remplace cette boucle par une boucle `Pour` **décroissante** qui donne le même résultat.

Algorithme : ch2 ← "" ; Pour i de long(ch)-1 à 0 (pas -1) Faire

ch2 ← ch2 + ch[i] ; Fin Pour

Python : ch2 = "" ; for i in range(len(ch)-1, -1, -1) :

ch2 = ch2 + ch[i]

II La boucle à condition d'arrêt : Répéter … Jusqu'à

DÉFINITION

La structure `Répéter … Jusqu'à` est utilisée quand le nombre de répétitions est **inconnu** et que le traitement doit être exécuté **au moins une fois**.

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

```
Répéter
      <traitement>
Jusqu'à <condition de sortie>
```

```
while True :
      <traitement>
      if condition_de_sortie :
            break

# ou bien :
<traitement>
while not condition_de_sortie :
      <traitement>
```

REMARQUE

- La condition est une **condition de sortie** : dès qu'elle est vérifiée, on quitte la boucle.
- Elle peut être simple ou composée.
- C'est la structure idéale pour le **contrôle de saisie** : elle oblige l'utilisateur à entrer une donnée valide.
- S'il y a un compteur, il faut l'**initialiser avant** la boucle.

LE RÉFLEXE À CONNAÎTRE PAR CŒUR

```
# Contrôle de saisie : n doit être entre 5 et 25
while True :
    n = int(input("n = "))
    if 5 < n < 25 :
        break
    print("Valeur refusée, recommence.")
```

ACTIVITÉ — 2 — TRACE D'EXÉCUTION

Soit l'algorithme suivant :

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

```
Algorithme Exercice
Début
    Lire(n)
    c ← 0
    k ← 2*n - 1
    Répéter
          c ← c + k
          k ← k - 2
    Jusqu'à (k < 1)
    Ecrire(c)
Fin
```

```
n = int(input())
c = 0
k = 2*n - 1
while True :
      c = c + k
      k = k - 2
      if k < 1 :
            break
print(c)
```

1) Donne les valeurs successives de c :

n	c_1	c_2	c_3	c_4	c_5	c final
3	*5*	*8*	*9*	*–*	*–*	*9*
5	*9*	*16*	*21*	*24*	*25*	*25*
7	*13*	*24*	*33*	*40*	*45*	*49*

2) Déduis-en le rôle de cet algorithme.

Cet algorithme calcule n^2 : c est la somme des n premiers entiers impairs (2n-1) + (2n-3) + … + 3 + 1.

Exemple : n = 3 → 5 + 3 + 1 = 9 = 3^2 (pour n = 7 il faut 7 tours : c final = 49).

III La boucle à condition d'entrée : Tant que … Faire

DÉFINITION

La structure `Tant que … Faire` est utilisée quand le nombre de répétitions est **inconnu** et que le traitement peut être exécuté **zéro fois**.

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`Tant que <condition d'entrée> Faire`
`  <traitement>`
`Fin Tant que`
	`while condition_d_entrée :`
	`  <traitement>`

REMARQUE

- La condition est une **condition d'entrée** : tant qu'elle est vraie, on itère encore.
- Elle peut être simple ou composée.
- S'il y a un compteur, il faut l'initialiser avant la boucle **et le faire progresser dedans**.

ERREUR FRÉQUENTE

Si la variable testée n'évolue jamais dans le corps de la boucle, le programme **tourne à l'infini**. Avant d'exécuter, demande-toi toujours : « qu'est-ce qui fera devenir cette condition fausse ? »

ASTUCE

Comment choisir ?
• Nombre de tours connu → `Pour`
• Au moins un tour obligatoire (saisie, menu) → `Répéter`
• Peut-être zéro tour (recherche, test) → `Tant que`

ACTIVITÉ — 3 — COMPTER LES CHIFFRES

Soit la séquence ci-dessous, où x est un entier naturel :

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`Lire(x)`
`nb ← 1`
`Tant que (x DIV 10 <> 0) Faire`
`  nb ← nb + 1`
`  x ← x DIV 10`
`Fin Tant que`
	`x = int(input())`
	`nb = 1`
	`while x // 10 != 0 :`
	`  nb = nb + 1`
	`  x = x // 10`

1) Complète le tableau par la valeur finale de nb :

x	5403	176	3
nb	*4*	*3*	*1*

2) Donne le rôle de cette séquence.

Cette séquence compte le nombre de chiffres nb de l'entier x (on divise x par 10 tant que le quotient est non nul).

Exemple : 5403 → 540 → 54 → 5 : 3 divisions, donc nb = 1 + 3 = 4.

3) Écris une séquence équivalente **sans** structure itérative.

Algorithme : nb ← Long(Convch(x))

Python : x = int(input())

nb = len(str(x))

Fiche mémo — Les structures itératives

Pour ... Faire	`for i in range(Vi, Vf+1)` — nombre de tours connu
Répéter ... Jusqu'à	condition de **sortie** — au moins un tour
Tant que ... Faire	condition d'**entrée** — peut-être zéro tour
Compteur	`c = 0` avant la boucle, `c = c + 1` dedans
Accumulateur	somme → initialiser à **0** · produit → initialiser à **1**
Drapeau	`trouve = False` puis `trouve = True` dès qu'on trouve
Boucle infinie	arrive quand la variable de la condition n'évolue pas dans la boucle

SÉRIE D'EXERCICES 3

Les structures de contrôle itératives

17 exercices classés par difficulté · Facile Moyen Défi

Exercice 1 Occurrence d'une lettre

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de chercher et d'afficher le nombre d'occurrences d'une lettre L dans une chaîne Ch.

EXEMPLE D'EXÉCUTION

```
Ch : Bonjour · L : o
Occurrence = 2
```

Exercice 2 Maximum d'un tableau

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de trouver et d'afficher le plus grand élément d'un vecteur T de n entiers.

EXEMPLE D'EXÉCUTION

```
n=8 · T : 2|5|8|1|0|9|5|7
Max = 9
```

Exercice 3 Produit et somme

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de déterminer le produit des éléments positifs PP et la somme des éléments négatifs SN d'un vecteur T de n entiers.

EXEMPLE D'EXÉCUTION

```
n=6 · T : 2|−5|−8|1|4|−3
PP = 8 et SN = −16
```

Exercice 4 Pairs et impairs

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de déterminer le nombre d'éléments pairs NP et impairs NI d'un vecteur T de n entiers.

EXEMPLE D'EXÉCUTION

```
n=7 · T : 2|5|8|1|4|3|9
NP = 3 et NI = 4
```

Exercice 5 Moyenne de la classe

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir les notes de n élèves (5 < n < 25) puis d'afficher la moyenne Moy.

EXEMPLE D'EXÉCUTION

```
n=8 · T : 11|2|5|8|13|9|19|15
Moy = 11.71
```

Exercice 6 Palindrome

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un mot M d'au plus 15 caractères et d'afficher s'il est palindrome.

Un mot est palindrome s'il se lit de la même façon dans les deux sens : RADAR, LAVAL, ELLE...

EXEMPLE D'EXÉCUTION

```
M : AZZA → AZZA est palindrome
M : AMIR → AMIR n'est pas palindrome
```

Exercice 7 Nombre premier

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un entier n différent de 1 et d'afficher s'il est premier.

EXEMPLE D'EXÉCUTION

```
n : 7 → 7 est premier
n : 10 → 10 n'est pas premier
```

Exercice 8 Nombre de chiffres

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un entier n et d'afficher son nombre de chiffres.

EXEMPLE D'EXÉCUTION

```
n : 2024
Le nombre de chiffres de 2024 est 4
```

Exercice 9 Miroir d'un nombre

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un entier n > 0 et d'afficher son miroir.

EXEMPLE D'EXÉCUTION

```
n : 123
Son miroir est : 321
```

Exercice 10 Somme des chiffres pairs

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un entier n strictement positif et d'afficher la somme s de ses chiffres pairs.

EXEMPLE D'EXÉCUTION

```
n : 54276
La somme des chiffres pairs de 54276 est 12
```

Exercice 11 Nombre parfait

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir un entier n (1 < n < 100) et d'afficher s'il est parfait.

Un nombre est parfait s'il est égal à la somme de ses diviseurs sauf lui-même : 6 = 1 + 2 + 3.

EXEMPLE D'EXÉCUTION

```
n : 6 → 6 est parfait
n : 10 → 10 n'est pas parfait
```

Exercice 12 Le juste prix

Défi

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de faire deviner à l'utilisateur un nombre entre 0 et 100 choisi par l'ordinateur, en comptant les coups et en indiquant « c'est plus » ou « c'est moins ».

EXEMPLE D'EXÉCUTION

```
60 → C'est moins
30 → C'est plus
40 → C'est moins
36 → Juste, en 5 coups !
```

Exercice 13 Rectangle plein

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet d'afficher un rectangle plein d'étoiles.

EXEMPLE D'EXÉCUTION

```
longueur=5 · largeur=4
*****
*****
*****
*****
```

Exercice 14 Rectangle creux

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet d'afficher un rectangle creux d'étoiles.

EXEMPLE D'EXÉCUTION

```
longueur=5 · largeur=4
*****
*   *
*   *
*****
```

Exercice 15 Triangle rectangle

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet d'afficher un triangle rectangle d'étoiles de hauteur h.

EXEMPLE D'EXÉCUTION

```
hauteur=5
*
**
***
****
*****
```

Exercice 16 Binaire → décimal

Moyen

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de convertir un entier écrit en binaire en sa valeur décimale.

EXEMPLE D'EXÉCUTION

```
10111010
Sa valeur en décimal est 186
```

Exercice 17 Nombre de voyelles

Facile

Ecris l'**algorithme** et son **implémentation en Python** d'un programme qui permet de saisir une chaîne Ch et d'afficher le nombre de voyelles qu'elle contient.

EXEMPLE D'EXÉCUTION

```
Ch : Python
Le nombre de voyelles = 1
```